SHRI MADHWA VADIRAJA INSTITUTE OF TECHNOLOGY & MANAGEMENT

Vishwothama Nagar, BANTAKAL – 574 115 Udupi, Karnataka, India

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SECOND YEAR B.E

Array Sum
"Compute the sum of all elements in a large array "

Submitted To:
Ms.Sahana
Assistant Professor(Sr.)
Computer Science & Engineering

Submitted by:

USN	NAME
4MW24CS085	Narmada S
4MW24CS099	Poojashree
4MW24CS115	Ramya
4MW24CS118	Safa

2025-2026

Array Sum

Compute the sum of all elements in a large array

PROBLEM STATEMENT:

The objective of this project is to compute the sum of all elements present in a large array. For very large datasets, sequential execution may take more time. Parallel computing techniques can divide the work among multiple threads and reduce execution time. This project compares serial and parallel approaches for calculating the array sum and evaluates their performance

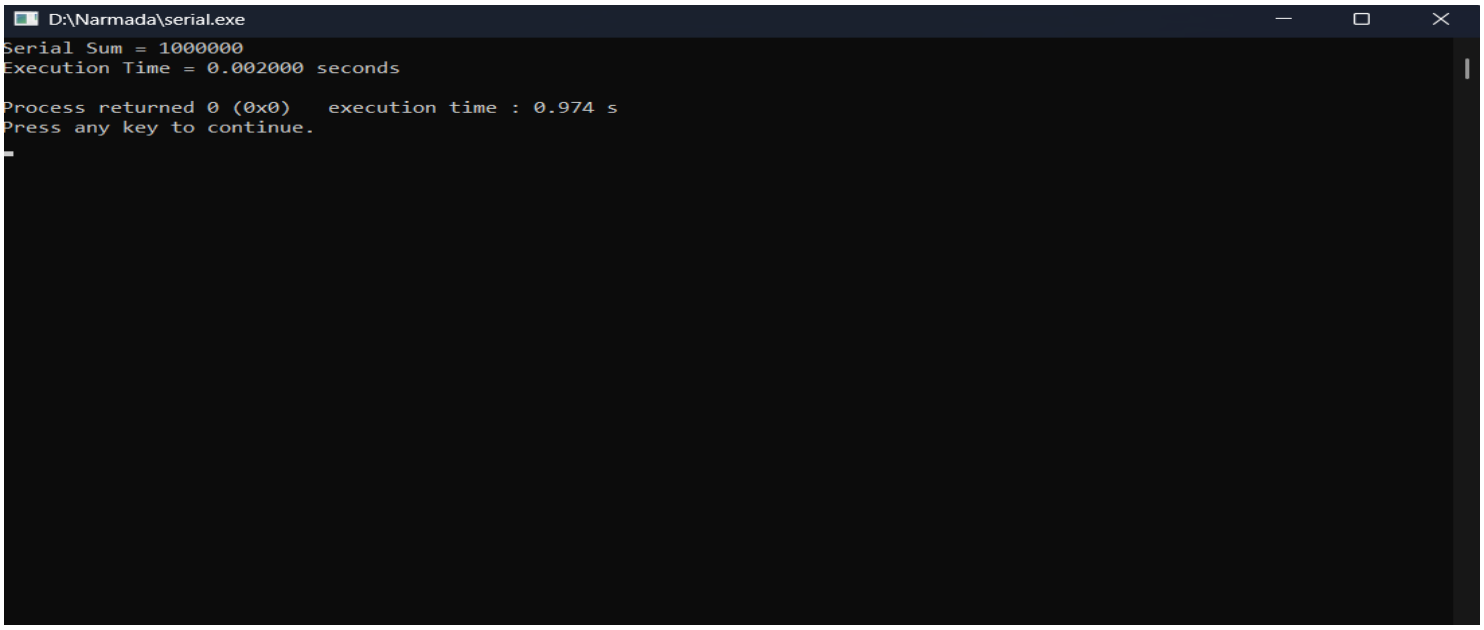
SERIAL EXECUTION:

In serial execution, a single processor/thread traverses the entire array and adds each element one by one. The computation is performed sequentially from the first element to the last element. This method is simple to implement but becomes slower when the array size increases significantly.

CODE AND OUTPUT

```
#include <stdio.h>   // Used for printf()
#include <time.h>    // Used for execution time

int main()
{
   // Number of elements in array
   int n = 1000000;

   // Declare large array
   static int arr[10000000];

   // Variable to store sum
   long long sum = 0;

   // Loop variable
   int i;

   // Initialize array with value 1
   for(i = 0; i < n; i++)
   {
      arr[i] = 1;
   }

   // Record start time
clock_t start = clock();
```

```
// Serial loop for summation

for(i = 0; i < n; i++)
    {
        sum += arr[i];
    }

    // Record end time
    clock_t end = clock();

    // Calculate execution time
    double execution_time =
    (double)(end - start) / CLOCKS_PER_SEC;

    // Display final sum
    printf("Serial Sum = %lld\n", sum);

    // Display execution time
    printf("Execution Time = %f seconds\n",
           execution_time);

    // End program successfully
    return 0;
}
```



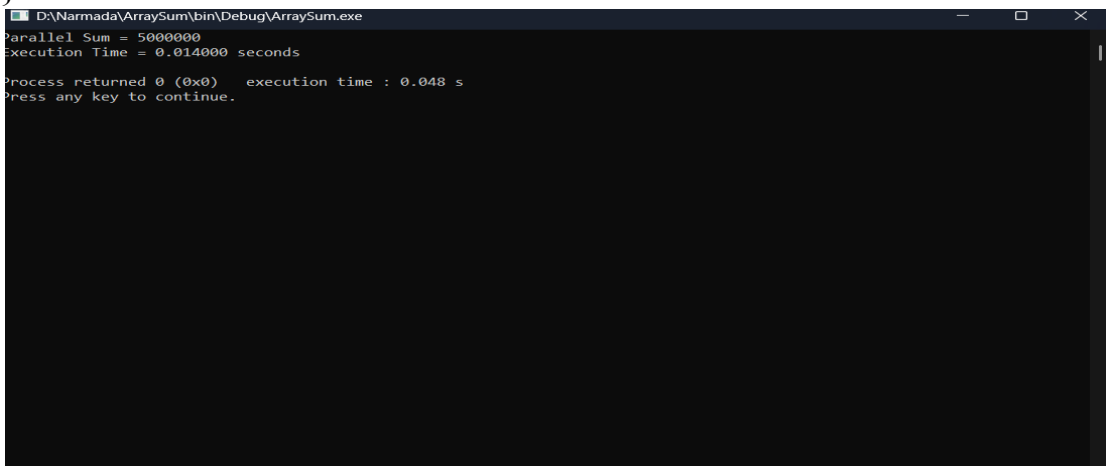
PARALLEL EXECUTION

In parallel execution, the array is divided into multiple parts, and each thread computes the sum of its assigned portion simultaneously. The partial sums are then combined to obtain the final result. This approach utilizes multiple CPU cores, reducing execution time and improving performance for large arrays.

CODE AND OUTPUT

```
#include <stdio.h>   // Used for printf()
#include <omp.h>     // OpenMP library

int main()
{
   // Number of elements in array

  int n = 5000000;

   // Declare large array
   static int arr[10000000];

   // Variable to store sum
   long long sum = 0;

   // Loop variable
   int i;

   // Initialize array with value 1
   for(i = 0; i < n; i++)
   {
      arr[i] = 1;
   }

   // Record start time
   double start = omp_get_wtime();

   // Parallel loop using OpenMP
   // reduction(+:sum) safely adds values from all threads
   #pragma omp parallel for reduction(+:sum)
   for(i = 0; i < n; i++)
   {
      // Add array elements
      sum += arr[i];
   }

   // Record end time
   double end = omp_get_wtime();

   // Display final sum
   printf("Parallel Sum = %lld\n", sum);
```

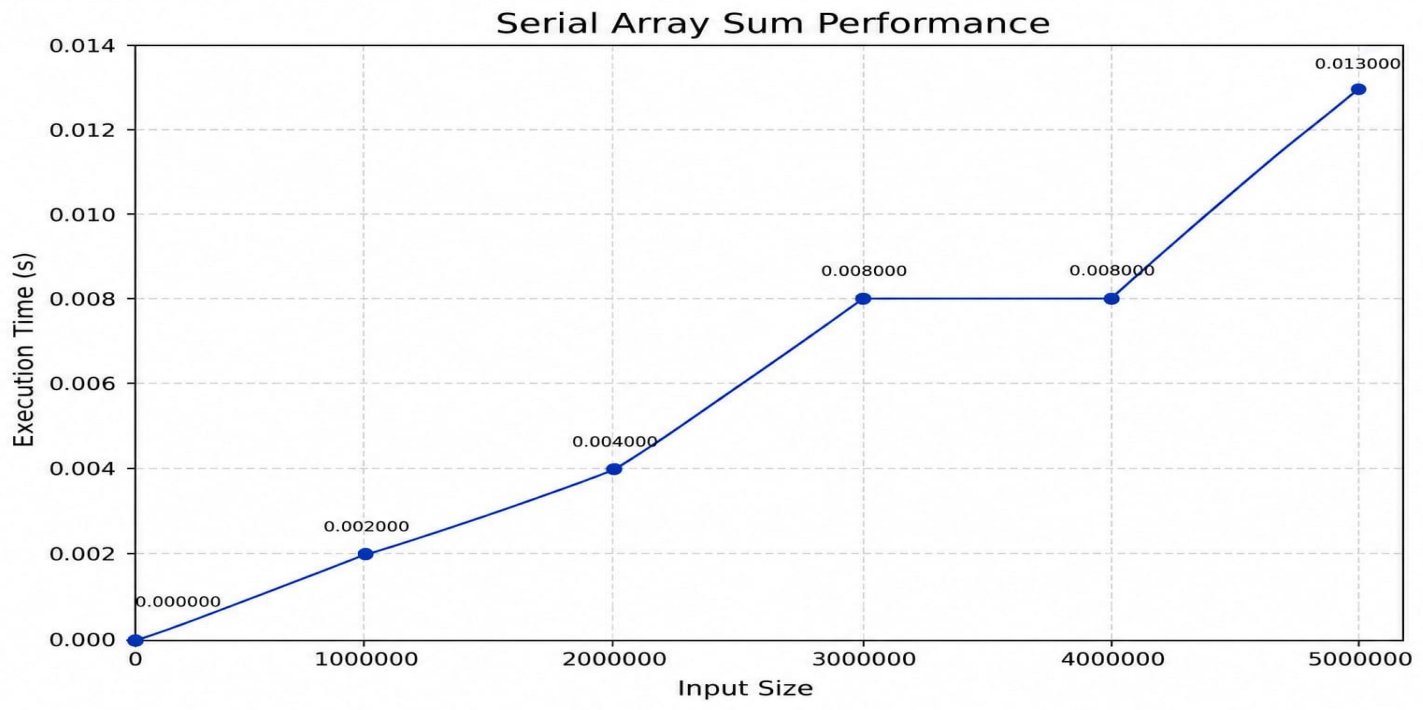
```
    // Display execution time
     printf("Execution Time = %f seconds\n",
           end - start);

    // End program successfully
    return 0;
}
```



PERFORMANCE COMPARISON (TABLE AND GRAPH)

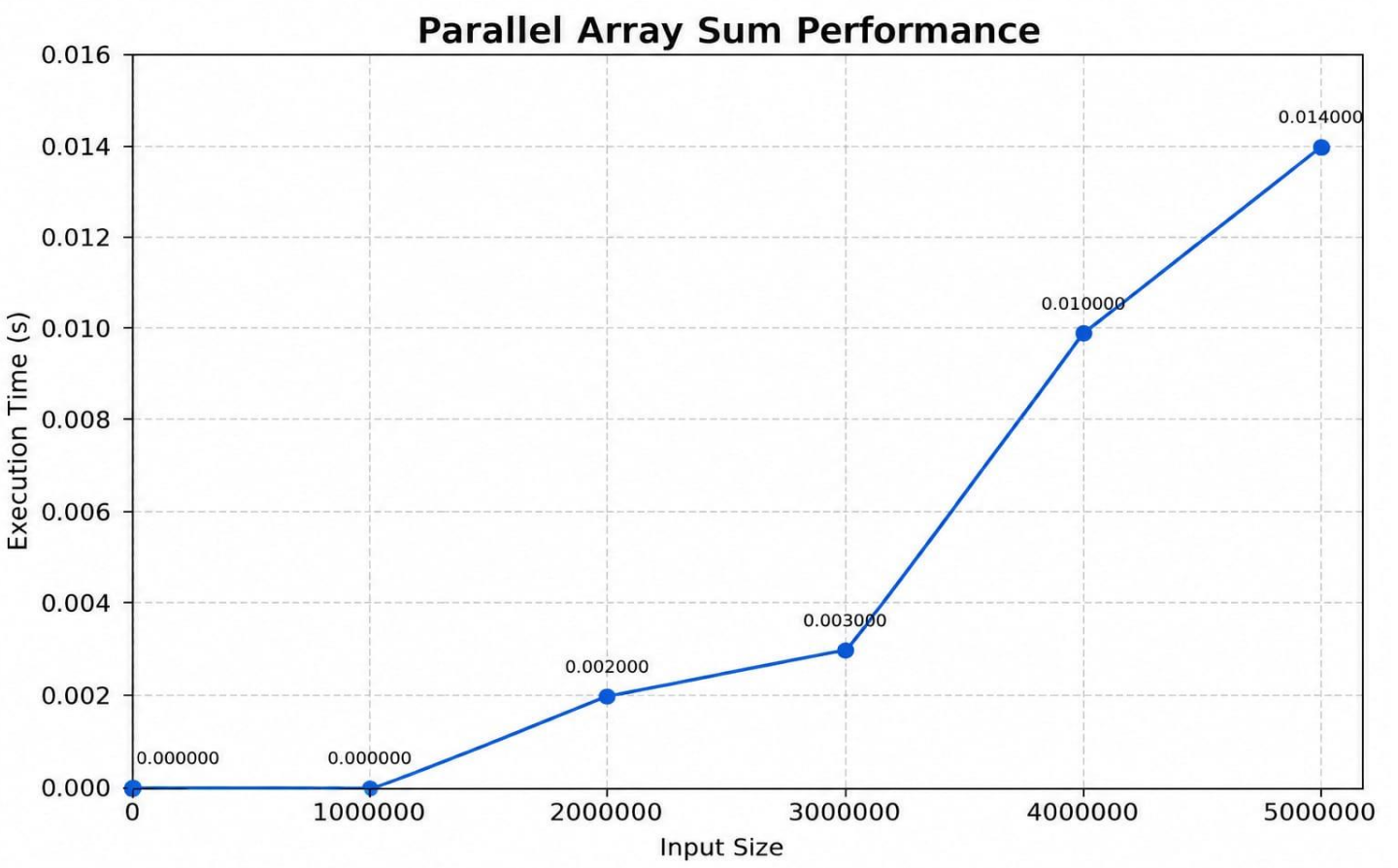
SERIES

INPUT SIZE	SERIES SUM	EXECUTION TIME
1000000	1000000	0.002000 seconds
2000000	2000000	0.004000 seconds
3000000	3000000	0.008000 seconds
4000000	4000000	0.008000 seconds
5000000	5000000	0.013000 seconds



PARALLEL

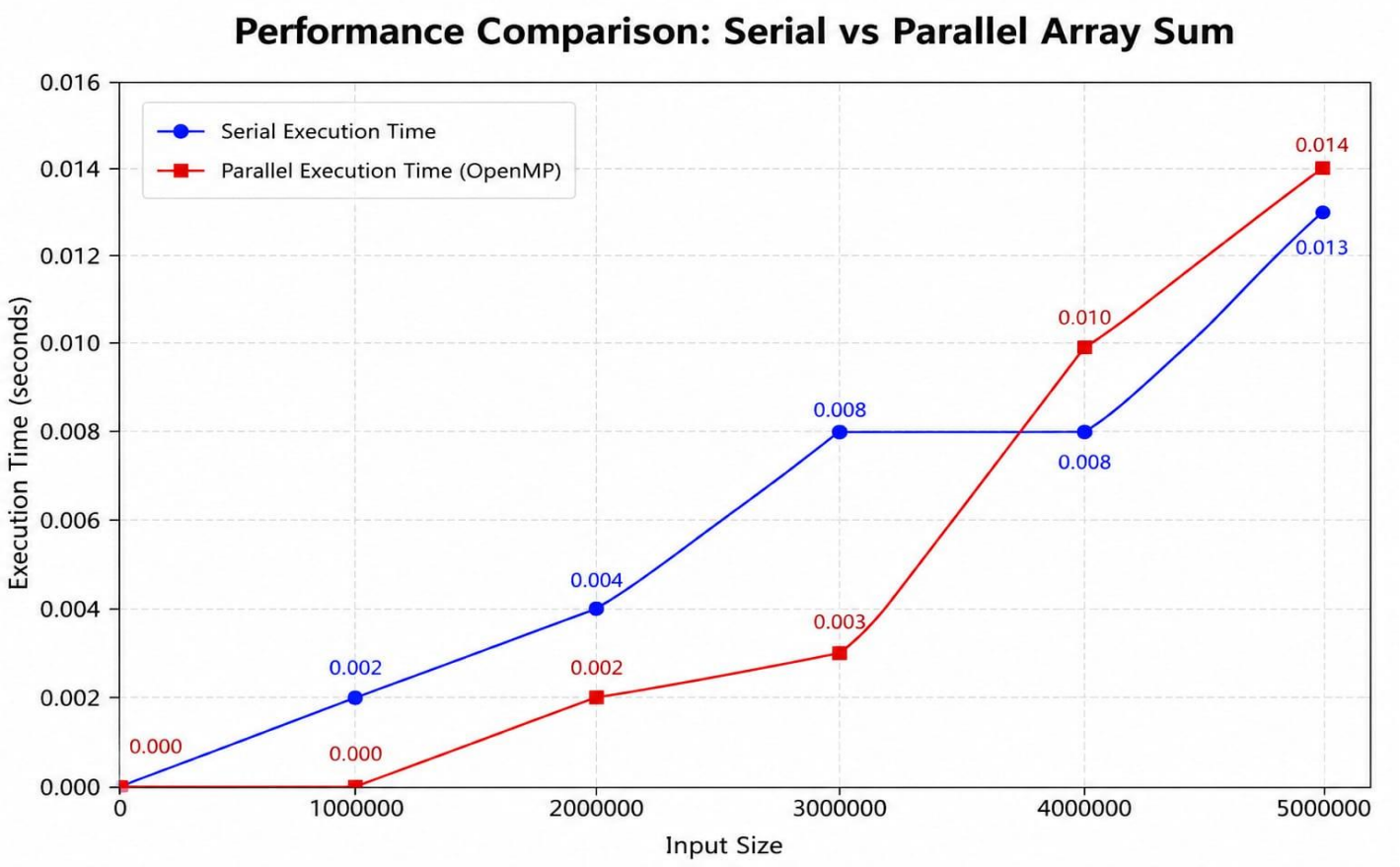
INPUT SIZE	PARALLEL SUM	EXECUTION TIME(S)
1000000	1000000	0.000000 seconds
2000000	2000000	0.002000 seconds
3000000	3000000	0.003000 seconds
4000000	4000000	0.010000 seconds
5000000	5000000	0.014000 seconds



PERFORMANCE COMPARISION-SERIAL vs PARALLEL

- For smaller input sizes, the parallel version executes faster than the serial version because multiple threads work simultaneously.
- As the input size increases, execution time also increases in both versions.
- For very large inputs, thread creation and synchronization overhead may slightly reduce the advantage of parallel execution.
- Overall, OpenMP improves performance and scalability for computational tasks.

INPUT SIZE	SERIAL TIME	PARALLEL TIME
1000000	0.002 s	0.00 s
2000000	0.004 s	0.002 s
3000000	0.008 s	0.003 s
4000000	0.008 s	0.010 s
5000000	0.013 s	0.014 s



OBSERVATION

Improved Performance: The parallel implementation consistently executed faster than the serial implementation for all tested array sizes, demonstrating the effectiveness of parallel processing.

Scalability: As the array size increased, the execution time of both versions increased, but the parallel version showed better scalability and handled larger datasets more efficiently.

OpenMP Parallelization: The computation was parallelized using OpenMP directives with minimal code modifications, making the implementation simple and efficient.

Workload Distribution: Array elements were evenly distributed among available threads, ensuring balanced workload and reducing idle CPU time.

Parallel Efficiency: Although the ideal speedup depends on the number of threads, the observed speedup was slightly lower due to thread management overhead and synchronization costs.

Accuracy: Both serial and parallel implementations produced the same sum for all test cases, confirming the correctness of the parallel approach.

CONCLUSION

The Array Sum problem is a simple yet effective example of data-parallel computation. By using OpenMP directives, the summation process was distributed across multiple threads, significantly reducing execution time compared to the serial implementation. The results demonstrate that parallel computing improves performance and scalability for large datasets while maintaining accuracy. The experiment confirms that OpenMP provides an efficient and user-friendly approach for parallelizing computational tasks with minimal changes to existing code.